

SHABD v2.0

Usage Guide

Python AI-Function Framework

Single file · Zero dependencies
MCP Tools · Resources · Prompts
Spell Chains · Hot Reload · Ollama

Contents

1. Installation & Quick Start
2. @app.spell — Tools
3. @app.resource — MCP Resources (NEW v2)
4. @app.prompt — MCP Prompts (NEW v2)
5. SpellImage & SpellFile (NEW v2)
6. Spell Chains / Pipelines (NEW v2)
7. Multi-Project Groups (NEW v2)
8. YAML Spells — Zero Python (NEW v2)
9. Hot Reload (NEW v2)
10. Security — Tokens & Scopes
11. CPM Integration (NEW v2)
12. HTTP Endpoints & Dashboard
13. MCP Stdio — Claude Desktop
14. Ollama Integration (NEW)
15. Plugin System & Redis
16. Cross-Language: Python + .NET
17. API Reference

1. Installation & Quick Start

SHABD is a single Python file. No pip install needed — just copy shabd.py into your project and import it.

Copy the file

```
cp shabd.py your_project/

# Both names work:
from shabd import SHABD # new canonical name
from shabd import Conjure # backward compatible
```

Your first server

```
from shabd import SHABD
import os

app = SHABD(
    name="my-tools",
    secret=os.environ.get("SHABD_SECRET", "dev-secret-32bytes-min!!"),
    require_auth=False, # dev mode
)

@app.spell
def add(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b

app.serve(port=8765)

# Dashboard: http://127.0.0.1:8765/dashboard
```

Claude Desktop (MCP mode)

```
# claude_desktop_config.json:
{
  "mcpServers": {
    "my-tools": {
      "command": "python",
      "args": ["app.py", "--mcp"]
    }
  }
}

# app.py — add this:
import sys
if "--mcp" in sys.argv:
    app.mcp_stdio()
```

```
else:  
    app.serve(port=8765)
```

2. @app.spell — Defining Tools

Register any Python function as an AI-callable tool. Type hints become the JSON schema automatically.

Basic spell

```
@app.spell
def greet(name: str) -> str:
    """Greet a user. Docstring becomes the description."""
    return f"Hello, {name}!"

# Call: POST /spells/greet body: {"name": "Alice"}
```

All parameters

Parameter	Description
name	Custom spell name (default: function name)
description	Override the docstring description
scopes	Required token scopes e.g. ["admin", "read"]
rate_limit	Max calls per minute per user
cache_ttl	Cache result for N seconds
timeout	Max execution time in seconds (default: 30)
retries	Auto-retry count on failure
tags	Tags: ["read", "destructive", "idempotent"]

Advanced examples

```
# Async + cache + rate limit
@app.spell(rate_limit=20, cache_ttl=300, tags=['read'])
async def search(query: str, limit: int = 10) -> dict:
    """Search docs. Cached 5 min."""
    return await db.search(query, limit)

# Streaming spell (SSE)
@app.spell
def countdown(n: int = 5):
    """Stream countdown. Use POST /stream/countdown"""
    for i in range(n, 0, -1):
        yield {"i": i}
        time.sleep(0.3)
    yield {"done": True}
```

```
# Context injection
@app.spell
def whoami(ctx: Context) -> dict:
    """Get caller identity."""
    return {"subject": ctx.subject, "scopes": ctx.scopes,
            "transport": ctx.transport}
```

3. @app.resource — MCP Resources (NEW v2)

Expose files, database records, or API data as MCP Resources. These appear in Claude Desktop's Resources tab. This feature was in FastMCP but missing in SHABD v1 — now fixed.

```
# URI template — {variable} becomes a parameter
@app.resource("/docs/{slug}", mime_type="text/markdown")
def get_doc(slug: str) -> str:
    """Return a documentation page."""
    return open(f"docs/{slug}.md").read()

# Async resource
@app.resource("db://users/{id}", mime_type="application/json")
async def get_user(id: str) -> dict:
    """Fetch user from database."""
    return await db.find_user(id)

# Binary resource (image, PDF, etc.)
@app.resource("files/{name}", mime_type="image/png")
def get_image(name: str) -> bytes:
    return open(f"assets/{name}", "rb").read()
```

HTTP Endpoints

- GET /resources — list all resources
- GET /resources?uri=/docs/api — read a specific resource
- MCP: resources/list — works in Claude Desktop
- MCP: resources/read — fetch by URI

4. @app.prompt — MCP Prompts (NEW v2)

Reusable prompt templates that appear in Claude Desktop's Prompts tab. Was in FastMCP but missing in SHABD v1 — now fixed.

```
# Return a string
@app.prompt("code_review")
def review_prompt(language: str = "python",
focus: str = "bugs") -> str:
    """Code review template."""
    return f"Review this {language} code for {focus}."

# Return a messages list (multi-turn)
@app.prompt("analyze")
def analyze_prompt(data: str, depth: str = "standard") -> list:
    """Analysis prompt with structured messages."""
    return [
        {"role": "user",
        "content": {"type": "text",
        "text": f"Analyze ({depth}): {data}"}}
    ]
```

HTTP Endpoints

- GET /prompts — list all prompts
- POST /prompts/code_review — execute with body: {"language": "python"}

5. SpellImage & SpellFile (NEW v2)

Return images or downloadable files from spells. Claude Desktop renders images inline and shows a download link for files. Was in FastMCP but missing in SHABD v1 — now fixed.

```
from shabd import SpellImage, SpellFile
import io, matplotlib.pyplot as plt

# Image return type
@app.spell
def make_chart(data: list) -> SpellImage:
    """Generate a bar chart PNG."""
    fig, ax = plt.subplots()
    ax.bar(range(len(data)), data)
    buf = io.BytesIO()
    plt.savefig(buf, format='png', dpi=150)
    plt.close()
    return SpellImage(data=buf.getvalue(),
        mime_type="image/png")

# File return type
@app.spell
def export_csv(rows: list) -> SpellFile:
    """Export data as a CSV download."""
    content = "\n".join(",".join(str(c) for c in row)
        for row in rows).encode()
    return SpellFile(data=content,
        name="export.csv",
        mime_type="text/csv")
```

6. Spell Chains / Pipelines (NEW v2)

Connect multiple spells with the pipe | operator. Output of spell N becomes the input of spell N+1. Unique to SHABD — not in FastMCP.

```
# Define the individual spells first
@app.spell
def search(query: str) -> dict:
    return {"results": db.search(query)}

@app.spell
def summarize(results: dict) -> str:
    return ", ".join(r["title"] for r in results["results"])

@app.spell
def translate(text: str, lang: str = "hi") -> str:
    return translator.translate(text, dest=lang)

# Create the chain – one line
app.chain("search | summarize | translate",
name="smart_search",
description="Search, summarize, and translate")

# Now callable as: POST /spells/smart_search
# body: {"query": "AI news", "lang": "hi"}
```

How data passes between steps

- If spell N returns a dict — it becomes keyword args for spell N+1
- If spell N returns anything else — it becomes the first positional arg
- The chain itself is also registered as a regular spell

7. Multi-Project Groups (NEW v2)

Namespace spells for multiple projects on one server. Unique to SHABD.

```
finance = app.group("finance")
hr = app.group("hr", scopes=["hr"])
@finance.spell
def calculate_gst(amount: float) -> dict:
    return {"gst": amount * 0.18}
# Registered as: finance__calculate_gst
@hr.spell # Only tokens with 'hr' scope can call this
def get_salary(emp_id: str) -> dict:
    return db.get_salary(emp_id)
# Registered as: hr__get_salary
```

8. YAML Spells — Zero Python (NEW v2)

Define REST API tools in config.yaml without writing any Python code. Unique to SHABD.

```
# config.yaml
yaml_spells:
  - name: get_weather
    description: "Get weather for a city"
    url: "https://wttr.in/{city}?format=j1"
    method: GET
    params:
      city: {type: string, required: true}
  - name: create_ticket
    url: "https://jira.company.com/rest/api/3/issue"
    method: POST
    auth_env: "JIRA_TOKEN" # reads from env var
    params:
      title: {type: string, required: true}
      project: {type: string, default: "PROJ"}
```

```
# Load in Python:
app = SHABD.from_config("config.yaml")

# Or:
cfg = ConfigLoader.load("config.yaml")
app = ConfigLoader.build_conjure(cfg)
```

9. Hot Reload (NEW v2)

Spells file reloads automatically when you save it. No server restart needed. Unique to SHABD.

```
# Pass hot_reload to serve()
app.serve(
    port=8765,
    hot_reload="my_spells.py",
    reload_fn=register_spells,
)

# my_spells.py must have a register function:
def register_spells(app):
    @app.spell
    def my_tool(x: int) -> int:
        return x * 2

# Save my_spells.py -> server reloads automatically
# Log shows: "Hot reload complete. Added: {my_tool}"
```

10. Security — Tokens & Scopes

```
# Issue a token
token = app.issue_token(
    subject="user-123",
    scopes=["read", "admin"],
    ttl=3600 # 1 hour
)

# Pass in HTTP header:
# Authorization: Bearer {token}

# Protect a spell with scopes
@app.spell(scopes=['admin'])
def delete_all() -> dict:
    """Only tokens with admin scope can call this."""
    ...

# Check scope inside a spell
@app.spell
def sensitive(ctx: Context) -> str:
    if not ctx.has_scope("finance"):
        raise ForbiddenError("finance scope required")
    return "secret data"
```

Setting	Meaning
<code>require_auth=True</code>	All calls require a valid token
<code>require_auth=False</code>	Dev mode — no auth needed
<code>scopes=["*"]</code>	Token has access to all spells
<code>SHABD_SECRET env</code>	Set secret via environment variable
<code>HMAC-SHA256</code>	Token signing algorithm
<code>ttl=3600</code>	Token expires after 1 hour

11. CPM Integration (NEW v2)

Generate a CPM-compatible YAML config for your server automatically. Unique to SHABD.

```
# Generate CPM config
yaml_str = app.cpm_config(base_url="https://my-app.com")
print(yaml_str)

# Or via HTTP (while server is running):
# GET http://localhost:8765/cpm-config
# GET http://localhost:8765/cpm-config?base_url=https://prod.com

# Output example:
# project:
#   name: my-tools
#   type: shabd
#   base_url: https://my-app.com
#   spells:
#     - name: add
#     endpoint: /spells/add
```

12. HTTP Endpoints & Dashboard

Endpoint	Description
GET /	Service info — spells count, version
GET /health	Liveness check
GET /manifest	All tools/resources/prompts
GET /openapi.json	OpenAPI 3.1 spec auto-generated
GET /metrics	p50/p95/p99 latency, counters
GET /dashboard	Live web dashboard
GET /calls?n=100	Recent call log
GET /cpm-config	CPM project YAML
GET /resources	List all MCP resources
GET /resources?uri=/x/y	Read a specific resource
GET /prompts	List all prompts
POST /prompts/{name}	Execute a prompt
POST /spells/{name}	Call any spell
POST /stream/{name}	SSE stream from a streaming spell
WS /ws	WebSocket transport
POST /replay/{trace_id}	Re-run any past call

Dashboard tabs

Tab	Contents
Overview	Live stats, latency bars, error counts
Spells	All registered spells and chains
Resources	All MCP resources with URI templates
Prompts	All prompts with arguments
Playground	Live spell testing — auto-generated form from schema
Recent Calls	Last 100 calls with Replay button

13. MCP Stdio — Claude Desktop

SHABD speaks the full MCP protocol over stdio. Tools, Resources, and Prompts all work in Claude Desktop.

MCP Method	What it does
<code>initialize</code>	Version negotiation
<code>tools/list</code>	All <code>@app.spell</code>
<code>tools/call</code>	Execute a spell
<code>resources/list</code>	All <code>@app.resource()</code> — NEW v2
<code>resources/read</code>	Fetch by URI — NEW v2
<code>prompts/list</code>	All <code>@app.prompt()</code> — NEW v2
<code>prompts/get</code>	Execute a prompt — NEW v2
<code>ping</code>	Liveness check

14. Ollama Integration (NEW)

Yes — SHABD integrates with local Ollama in two ways:

Way 1: Spell calls Ollama internally

A SHABD spell makes an HTTP call to Ollama's local API. Ollama runs at <http://localhost:11434> by default.

```
import urllib.request, json

@app.spell
def ask_ollama(prompt: str, model: str = "llama3.2") -> str:
    """Send a prompt to local Ollama and get a response."""
    payload = json.dumps({
        "model": model,
        "prompt": prompt,
        "stream": False
    }).encode()
    req = urllib.request.Request(
        "http://localhost:11434/api/generate",
        data=payload,
        headers={"Content-Type": "application/json"},
        method="POST"
    )
    with urllib.request.urlopen(req, timeout=60) as r:
        result = json.loads(r.read())
        return result["response"]

@app.spell
def classify_trade(description: str) -> dict:
    """Use Ollama to classify a trade description."""
    prompt = f"""Classify this trade as BUY or SELL.
    Return JSON only: {{"action": "BUY"|"SELL", "confidence": 0-1}}
    Trade: {description}"""
    raw = ask_ollama(prompt, model="llama3.2")
    return json.loads(raw)
```

Way 2: Ollama agent calls SHABD tools

Run SHABD as an HTTP server. Then write a Python script that uses Ollama's chat API with your tools.

```
# Install: pip install ollama
import ollama, requests

# Get SHABD tool definitions
manifest = requests.get("http://localhost:8765/manifest").json()

# Convert to Ollama tool format
tools = [
```

```

{
  "type": "function",
  "function": {
    "name": s["name"],
    "description": s["description"],
    "parameters": s["input_schema"]
  }
}
for s in manifest["spells"]
]

# Chat with Ollama using SHABD tools
response = ollama.chat(
  model="llama3.2",
  messages=[
    {"role": "user", "content": "Add 5 and 7"}
  ],
  tools=tools
)

# If Ollama wants to call a tool:
if response.message.tool_calls:
  for call in response.message.tool_calls:
    result = requests.post(
      f"http://localhost:8765/spells/{call.function.name}",
      json=call.function.arguments
    ).json()
    print(f"Tool result: {result['result']}")

```

Ollama Models that support tool calling

- llama3.2 — recommended, good tool calling
- llama3.1 — also supports tools
- mistral — supports tools via Mistral format
- qwen2.5 — good for structured output
- gemma2 — basic tool support
- Install any: `ollama pull llama3.2`

Quick Ollama + SHABD setup

- 1. Start Ollama: `ollama serve`
- 2. Pull a model: `ollama pull llama3.2`
- 3. Start SHABD: `python app.py (HTTP mode)`

- 4. Run agent: `python ollama_agent.py`

15. Plugin System & Redis

```
# Custom plugin
class MyPlugin(ConjurePlugin):
    def on_call_end(self, ctx, spell_name, args,
                    result, error, elapsed_ms):
        send_to_datadog(spell_name, elapsed_ms, error is not None)

app = SHABD('my-tools', plugins=[MyPlugin()])

# Redis – distributed rate limiting + cache
from shabd import RedisPlugin

app = SHABD('my-tools',
            plugins=[RedisPlugin('redis://localhost:6379')])

# MCPClient – proxy any external MCP server
from shabd import MCPClient

client = MCPClient("github",
                  transport="stdio",
                  command="npx",
                  args=["-y", "@modelcontextprotocol/server-github"],
                  env={"GITHUB_TOKEN": "ghp_..."})

client.register_on(app) # creates 'github__create_issue' etc.
```

16. Cross-Language: Python + .NET

MCP is a protocol, not a language. Python SHABD and a .NET MCP server can communicate in three ways — all tested and working.

Method	How it works
Claude Desktop	Add both servers to <code>claude_desktop_config.json</code> . Claude uses both simultaneously in one conversation. Zero code changes.
MCPClient bridge	SHABD wraps .NET as spells: <code>MCPClient("risk", transport="http", url="http://localhost:5000/mcp").register_on(app)</code> — .NET tools become SHABD spells.
HTTP REST	Any language can call SHABD's HTTP API directly: POST <code>http://localhost:8765/spells/spell_name</code> with JSON body.

Issues that are already handled

- SSE responses: MCPClient parses .NET's SSE format automatically
- Mcp-Session-Id header: captured and resent automatically
- Protocol version: negotiated in initialize handshake
- Auth tokens: SHABD HMAC vs .NET JWT — separate servers, no conflict

17. API Reference

SHABD / Conjure class

Method / Decorator	Description
<code>SHABD(name, secret, ...)</code>	Create app. Both SHABD and Conjure work.
<code>@app.spell(...)</code>	Register a function as an AI tool.
<code>@app.resource(uri, ...)</code>	Register an MCP Resource. NEW v2.
<code>@app.prompt(name, ...)</code>	Register an MCP Prompt. NEW v2.
<code>app.chain(pipeline, name=...)</code>	Create a spell pipeline. NEW v2.
<code>app.group(name, ...)</code>	Create a ProjectGroup. NEW v2.
<code>app.cpm_config(base_url)</code>	Generate CPM YAML config. NEW v2.
<code>app.serve(port, hot_reload=...)</code>	Start HTTP server. hot_reload NEW v2.
<code>app.mcp_stdio()</code>	Start MCP stdio for Claude Desktop.
<code>app.invoke(spell_name, args)</code>	Call a spell in-process.
<code>app.issue_token(subject, scopes, ttl)</code>	Issue an HMAC token.
<code>app.manifest()</code>	Universal AI manifest dict.
<code>SHABD.from_config(path)</code>	Create app from YAML/JSON config.

Data classes

Class	Description
<code>SpellImage(data, mime_type, alt_text)</code>	Image return type. Has <code>.to_base64()</code> .
<code>SpellFile(data, name, mime_type)</code>	File return type. Has <code>.to_base64()</code> .
<code>Context</code>	Request context: <code>trace_id</code> , <code>subject</code> , <code>scopes</code> , <code>transport</code> , <code>elapsed_ms()</code>
<code>ConjurePlugin</code>	Base class for plugins. Override <code>on_call_start</code> , <code>on_call_end</code> , etc.
<code>RedisPlugin(url)</code>	Distributed rate limiting and cache via Redis.
<code>MCPClient(name, ...)</code>	Proxy any external MCP server as spells.